ADNI Knowledge Graph Design Blueprint v2

Revised per Advisor Feedback — Hybrid Architecture Edition
Disease Classification via RDF/SPARQL · Composite Unique Keys · Dynamic Graph

Authors: O. Gungor, S. N. Turhan, O. Pinarer, S. Arib, H. Baazaoui, R. Bouhamoum
Institutions: Galatasaray University · CY Cergy Paris University (ETIS Lab)
Dataset: ADNI (108 tables, ~5,800 columns, ~407K nodes, ~1.16M relationships)
Architecture: Hybrid — Neo4j (data + in-place ontology) + SPARQL (disease classification)

Changes from v1 (per Prof. Turhan's feedback):

1. Hybrid Architecture for Disease Classification: Diagnosis nodes now connect to ICD-10 via RDF/SPARQL (BioPortal endpoint + local rdflib). All other nodes remain in-place Neo4j ontology. This demonstrates dual competency and enables hierarchical disease reasoning (e.g., 'AD IS_A Degenerative Disease of Nervous System').

2. Composite Unique Constraints: Added to all observation nodes — CognitiveAssessment (visit_id + test_name), CSFBiomarker (visit_id + assay), BloodBiomarker (visit_id + analyte + assay), VolumetricMeasure (visit_id + region_name + hemisphere). Prevents duplicate nodes even if CREATE is used instead of MERGE.

3. Dynamic Graph with Change Detection: Hash-based row comparison (SHA-256) detects changes before insert. Batch audit trail (BatchIngestion nodes) tracks every ingestion run. This is what makes the graph 'dynamic' — continuous data acceptance without duplicates. Addresses Dr. Baazaoui's perspective on dynamic graph behavior.

Contents

1. Architecture Decision: Hybrid Neo4j + SPARQL

The thesis adopts a **hybrid architecture**: most ontological enrichment happens in-place within Neo4j (adding SNOMED, LOINC, UBERON codes as properties), but **disease classification specifically** connects to external knowledge bases (ICD-10, MONDO) via RDF/SPARQL. This is not arbitrary — disease classification hierarchies are deep, well-maintained, and benefit from formal subsumption reasoning that SPARQL provides natively.

Component	Technology	Scope	Justification
Patient data, visits, biomarkers, imaging, genetics	Neo4j (in-place ontology)	All nodes except disease hierarchy	Performance, existing pipeline, LPG-native
Disease classification hierarchy reasoning	RDF/SPARQL (BioPortal + rdflib)	Diagnosis nodes only	ICD-10 hierarchy, rdfs:subClassOf reasoning, WHO standard
Literature knowledge	Neo4j (SAME_AS bridge)	AlzKB subset (~200 concepts)	Already in Neo4j format (Romano 2024)
Causal relationships	Neo4j (native CAUSES edges)	Discovered edges from Phase 2	Performance for graph traversal

Why not RDF/SPARQL for everything? Our data is already in Neo4j (407K nodes). Migrating to a triple store would mean rewriting the entire pipeline. The hybrid approach keeps Neo4j's performance for patient-level queries while gaining SPARQL's reasoning for the one domain that truly needs it: hierarchical disease classification. This statement — 'You do NOT need to switch to RDF/SPARQL *except for disease classification*' — is more precise and defensible than the v1 claim.

2. LPG vs KG — Updated Comparison

Aspect	LPG (Current State)	True KG (Target State)
Schema	Implicit, application-defined	Formal ontology references (hybrid: properties + SPARQL)
Disease reasoning	dx_label = 'AD' (string)	ICD-10 G30.9 via SPARQL: AD IS_A G30 IS_A G20-G26
Biomarker semantics	Column names only	LOINC codes as properties
Brain regions	FreeSurfer labels	UBERON codes as properties
Interoperability	System-specific	ICD-10 (WHO) + SNOMED-CT + LOINC + UBERON + HPO
Inference	Not supported	SPARQL: hierarchical disease Neo4j: IS_A on OntologyConcept
Duplicate prevention	Manual checks	Composite unique constraints on all observation nodes

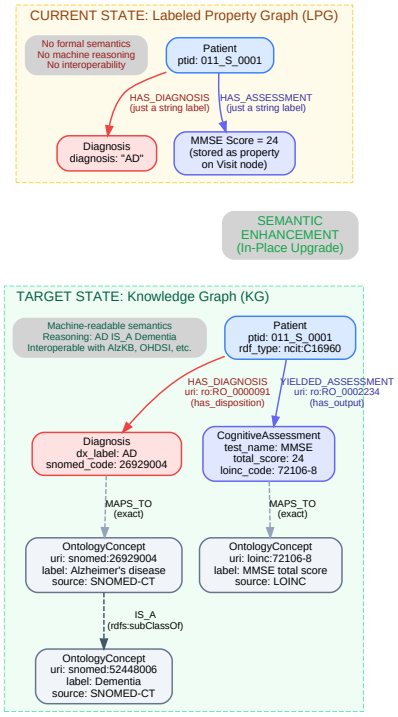

Figure 1: LPG vs Knowledge Graph — Side-by-side comparison. In the KG, every entity carries ontology type references and relationships have semantic URIs.

3. Target KG Schema — Complete Node & Relationship Diagram

Patient remains the hub node. Time is stored as properties on Visit nodes. Dashed lines = MAPS_TO ontology connections. Solid lines = data relationships. Diagnosis nodes additionally connect to ICD-10 via SPARQL (shown in Section 8).

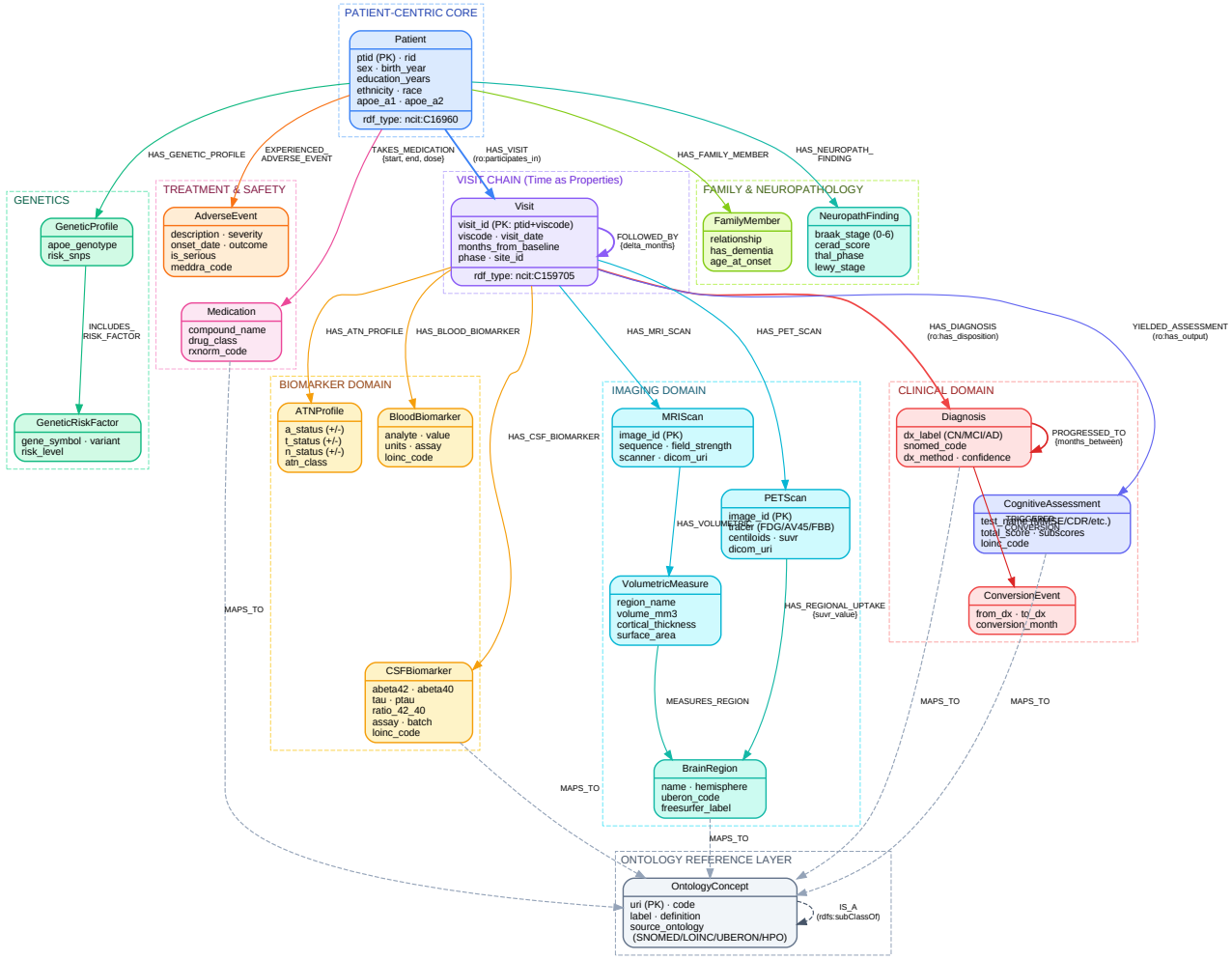

Figure 2: Complete Target KG Schema — 17 node types, 20+ relationship types, ontology reference layer. Diagnosis nodes are the bridge to the SPARQL world.

4. Node Catalog — Updated with Composite Constraints

Key change from v1: All observation nodes now have composite unique constraints to prevent duplicates. The red text highlights new/changed constraints.

Node Label	Key Properties	Ontology	Constraints & Indexes
Patient	ptid (PK), rid, sex, birth_year, education_years, ethnicity, race, apoe_a1/a2	ncit:C16960	**Unique: ptid** Index: rid
Visit	visit_id (PK: ptid+viscode), viscode, visit_date, months_from_baseline, phase, site_id	ncit:C159705	**Unique: visit_id** Composite idx: (ptid, viscode)
Diagnosis	dx_label (CN/MCI/AD), icd10_code, snomed_code, mondo_code, dx_method, confidence	ICD-10 via SPARQL + SNOMED in-place + MONDO in-place	**Unique: (visit_id, dx_label)** Idx: icd10_code
CognitiveAssessment	test_name, total_score, subscores (JSON), loinc_code, assessment_id	LOINC: 72106-8 (MMSE), etc.	**Unique: (visit_id, test_name)** Idx: loinc_code
CSFBiomarker	abeta42, abeta40, tau, ptau, ratio_42_40, assay, batch, loinc_code	LOINC: 72333-6	**Unique: (visit_id, assay)** Idx: assay
BloodBiomarker	analyte, value, units, assay, loinc_code	LOINC per analyte	**Unique: (visit_id, analyte, assay)**
ATNProfile	a_status, t_status, n_status, atn_class	NIA-AA Framework	**Unique: (visit_id)** Idx: atn_class
MRIScan	image_id (PK), sequence, field_strength, scanner	DICOM + SNOMED	**Unique: image_id**
PETScan	image_id (PK), tracer, centiloids, suvr	SNOMED: 82918005	**Unique: image_id** Idx: tracer
VolumetricMeasure	region_name, volume_mm3, cortical_thickness, surface_area, hemisphere	UBERON codes	**Unique: (visit_id, region_name, hemisphere)**
BrainRegion	name, hemisphere, uberon_code, freesurfer_label	UBERON: 0002421	**Unique: (name, hemisphere)**
GeneticProfile	apoe_genotype, risk_snps	Gene Ontology	Idx: apoe_genotype
Medication	compound_name, drug_class, rxnorm_code	RxNorm / ATC	Idx: compound_name
AdverseEvent	description, severity, onset_date, outcome, meddra_code	MedDRA codes	Idx: severity
FamilyMember	relationship, has_dementia, age_at_onset	HPO: HP:0002354	Idx: relationship

Node Label	Key Properties	Ontology	Constraints & Indexes
NeuropathFinding	braak_stage (0-6), cerad_score, thal_phase, lewy_stage	SNOMED neuropath	Idx: braak_stage
OntologyConcept	uri (PK), code, label, source_ontology, definition	Self-referencing (IS_A hierarchy)	**Unique: uri** Idx: code

5. Relationship Catalog

Relationship Type	Source -> Target	Properties	Ontology URI
HAS_VISIT	Patient -> Visit	—	ro:RO_0000056
FOLLOWED_BY	Visit -> Visit	delta_months	time:intervalBefore
HAS_DIAGNOSIS	Visit -> Diagnosis	source_table	ro:RO_0000091
PROGRESSED_TO	Diagnosis -> Diagnosis	months_between	ro:RO_0002263
TRIGGERED_CONVERSION	Diagnosis -> ConversionEvent	—	Custom
YIELDED_ASSESSMENT	Visit -> CognitiveAssessment	—	ro:RO_0002234
HAS_CSF_BIOMARKER	Visit -> CSFBiomarker	—	ro:RO_0002234
HAS_BLOOD_BIOMARKER	Visit -> BloodBiomarker	—	ro:RO_0002234
HAS_ATN_PROFILE	Visit -> ATNProfile	—	adni:derived_from
HAS_MRI_SCAN	Visit -> MRIScan	—	ro:RO_0002234
HAS_PET_SCAN	Visit -> PETScan	—	ro:RO_0002234
HAS_VOLUMETRIC	MRIScan -> VolumetricMeasure	—	ro:RO_0002234
MEASURES_REGION	VolumetricMeasure -> BrainRegion	—	bfo:BFO_0000066
HAS_GENETIC_PROFILE	Patient -> GeneticProfile	—	ro:RO_0000053
TAKES_MEDICATION	Patient -> Medication	start, end, dose	ro:RO_0000056
MAPS_TO	Any -> OntologyConcept	mapping_confidence	skos:exactMatch
IS_A	OntologyConcept -> OntologyConcept	—	rdfs:subClassOf
CLASSIFIED_AS	Diagnosis -> ICD-10 (via SPARQL)	icd10_code, mapping_source	skos:exactMatch (cross-system)
CAUSES (Phase 2)	Any -> Any	algorithm, p_value, effect_size	ro:RO_0002411

New in v2: CLASSIFIED_AS relationship connects Diagnosis nodes to ICD-10 codes resolved via SPARQL. This is the bridge between Neo4j and the RDF world.

6. Index & Constraint Strategy — Updated

Uniqueness Constraints (updated with composite keys):

```
// === Core Entity Constraints ===
CREATE CONSTRAINT patient_ptid FOR (p:Patient) REQUIRE p.ptid IS UNIQUE;
CREATE CONSTRAINT visit_id FOR (v:Visit) REQUIRE v.visit_id IS UNIQUE;
CREATE CONSTRAINT mri_image_id FOR (m:MRIScan) REQUIRE m.image_id IS UNIQUE;
CREATE CONSTRAINT pet_image_id FOR (p:PETScan) REQUIRE p.image_id IS UNIQUE;
CREATE CONSTRAINT brain_region FOR (b:BrainRegion)
REQUIRE (b.name, b.hemisphere) IS UNIQUE;
CREATE CONSTRAINT ontology_uri FOR (o:OntologyConcept) REQUIRE o.uri IS UNIQUE;

// === NEW: Composite Keys for Observation Nodes ===
// Prevents duplicates when same CSV is processed twice
CREATE CONSTRAINT assess_unique FOR (c:CognitiveAssessment)
REQUIRE (c.visit_id, c.test_name) IS UNIQUE;
CREATE CONSTRAINT csf_unique FOR (c:CSFBiomarker)
REQUIRE (c.visit_id, c.assay) IS UNIQUE;
CREATE CONSTRAINT blood_unique FOR (b:BloodBiomarker)
REQUIRE (b.visit_id, b.analyte, b.assay) IS UNIQUE;
CREATE CONSTRAINT vol_unique FOR (v:VolumetricMeasure)
REQUIRE (v.visit_id, v.region_name, v.hemisphere) IS UNIQUE;
CREATE CONSTRAINT atn_unique FOR (a:ATNProfile)
REQUIRE (a.visit_id) IS UNIQUE;
CREATE CONSTRAINT dx_unique FOR (d:Diagnosis)
REQUIRE (d.visit_id, d.dx_label) IS UNIQUE;
```

Why composite keys matter: Even with MERGE, if the pipeline's `create_observation()` function uses CREATE for speed, composite unique constraints act as the **last line of defense** against duplicates. If the same CSV is ingested twice, the constraint catches it at database level.

Performance Indexes:

```
CREATE INDEX patient_rid FOR (p:Patient) ON (p.rid);
CREATE INDEX visit_ptid FOR (v:Visit) ON (v.ptid);
CREATE INDEX visit_viscode FOR (v:Visit) ON (v.viscode);
CREATE INDEX visit_phase FOR (v:Visit) ON (v.phase);
CREATE INDEX dx_label FOR (d:Diagnosis) ON (d.dx_label);
CREATE INDEX dx_snomed FOR (d:Diagnosis) ON (d.snomed_code);
CREATE INDEX dx_icd10 FOR (d:Diagnosis) ON (d.icd10_code); // NEW
CREATE INDEX assess_test FOR (c:CognitiveAssessment) ON (c.test_name);
CREATE INDEX assess_loinc FOR (c:CognitiveAssessment) ON (c.loinc_code);
CREATE INDEX atn_class FOR (a:ATNProfile) ON (a.atn_class);
CREATE INDEX pet_tracer FOR (p:PETScan) ON (p.tracer);
CREATE INDEX vol_region FOR (v:VolumetricMeasure) ON (v.region_name);
CREATE INDEX med_name FOR (m:Medication) ON (m.compound_name);
CREATE INDEX gene_sym FOR (g:GeneticRiskFactor) ON (g.gene_symbol);
CREATE INDEX ontology_code FOR (o:OntologyConcept) ON (o.code);
CREATE INDEX ontology_src FOR (o:OntologyConcept) ON (o.source_ontology);
```

7. Dynamic Graph: Change Detection & Audit Trail

The graph must behave as a **living, dynamic system** — accepting new data continuously without rebuilding. This was a key point raised by Dr. Baazaoui. The solution has three layers:

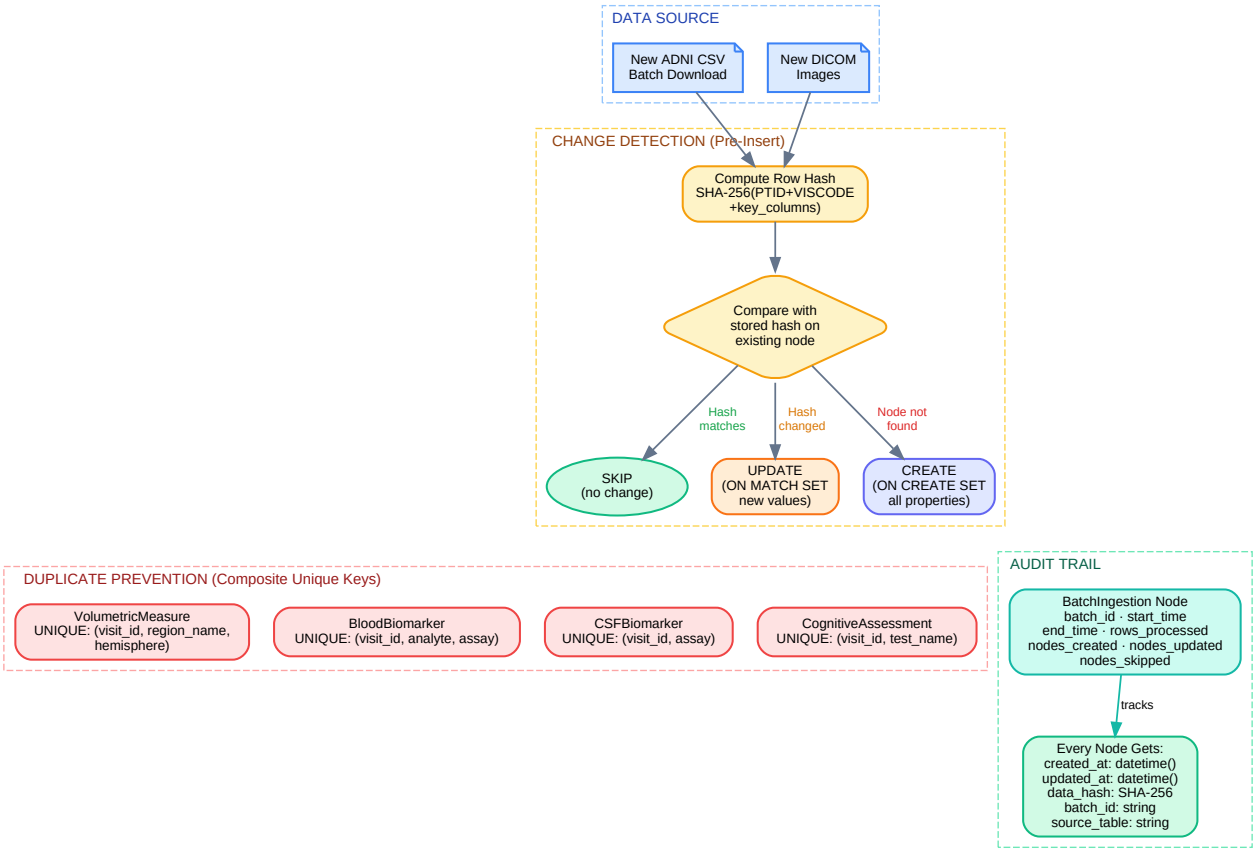

Figure 3: Dynamic graph architecture — hash-based change detection, composite unique constraints as safety net, and batch audit trail for traceability.

Layer 1: Hash-Based Change Detection

```
import hashlib

def compute_row_hash(row: dict, key_columns: list) -> str:
    """SHA-256 of key columns for change detection."""
    values = '|'.join(str(row.get(c, '')) for c in key_columns)
    return hashlib.sha256(values.encode()).hexdigest()

# In the MERGE query:
# MERGE (c:CognitiveAssessment {visit_id: $vid, test_name: $tn})
# ON CREATE SET c.data_hash = $hash, c.created_at = datetime()
# ON MATCH SET
#   CASE WHEN c.data_hash <> $hash THEN
#     c.total_score = $score, c.data_hash = $hash,
#     c.updated_at = datetime()
#   END
```

Layer 2: Composite Unique Constraints

Even if CREATE is used (for performance in bulk inserts), the database-level composite unique constraints from Section 6 will reject duplicates with a constraint violation error. The pipeline catches this error and logs it as 'skip (duplicate)'.

Layer 3: Batch Audit Trail

```
// BatchIngestion meta-node — one per ingestion run
CREATE (b:BatchIngestion {
  batch_id: $batch_id,
  source_table: $table_name,
  start_time: datetime(),
  rows_processed: $count,
  nodes_created: $created,
  nodes_updated: $updated,
  nodes_skipped: $skipped
})
```

Result: The graph can be re-run on the same data any number of times. Only genuinely new or changed records produce modifications. Every batch is logged. This is what makes the graph 'dynamic' per Dr. Baazaoui's requirement.

8. Disease Classification via SPARQL

This section implements Prof. Turhan's feedback: disease definitions specifically should connect to ICD-10 via RDF/SPARQL, while all other semantic enrichment stays in Neo4j.

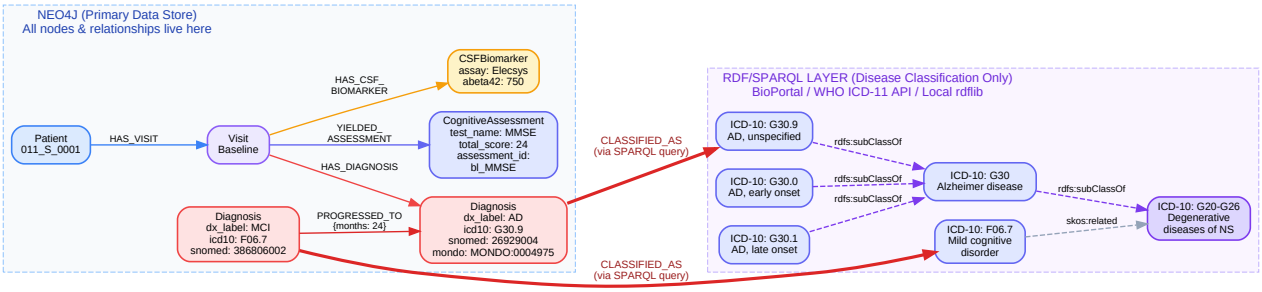

Figure 4: Hybrid architecture — Neo4j stores all patient data. Diagnosis nodes carry icd10_code properties AND connect to the ICD-10 hierarchy via SPARQL queries to BioPortal.

ICD-10 Code Mapping for ADNI Diagnoses

ADNI dx_label	ICD-10 Code	ICD-10 Label	SNOMED-CT Code	MONDO Code
AD	G30.9	Alzheimer disease, unspecified	26929004	MONDO:0004975
AD (early onset)	G30.0	Alzheimer disease with early onset	26929004	MONDO:0004975
AD (late onset)	G30.1	Alzheimer disease with late onset	26929004	MONDO:0004975
MCI	F06.7	Mild cognitive disorder	386806002	MONDO:0024647
CN (Normal)	Z03.89	No diagnosis or condition	17621005	—
Dementia (other)	F03.9	Unspecified dementia	52448006	MONDO:0001627

Implementation: SPARQL Query to BioPortal

```
from rdflib import Graph, Namespace
from SPARQLWrapper import SPARQLWrapper, JSON

# Option 1: BioPortal SPARQL endpoint
sparql = SPARQLWrapper(
  'https://sparql.bioontology.org/sparql')
sparql.addParameter('apikey', BIOPORTAL_KEY)

# Query: Get ICD-10 hierarchy for G30.9
sparql.setQuery("""
  PREFIX rdfs:
  SELECT ?parent ?parentLabel WHERE {

      rdfs:subClassOf ?parent .
    ?parent rdfs:label ?parentLabel .
  }
""")
results = sparql.query().convert()
```

```
# Returns: G30.9 -> G30 -> G30-G32 -> G20-G26

# Option 2: Local rdflib for offline use
g = Graph()
g.parse('icd10cm_subset.ttl', format='turtle')
# Query locally with the same SPARQL
```

Storing the Result in Neo4j

```
// Step 1: Add ICD-10 codes to Diagnosis nodes
MATCH (d:Diagnosis) WHERE d.dx_label = 'AD'
SET d.icd10_code = 'G30.9',
    d.snomed_code = '26929004',
    d.mondo_code = 'MONDO:0004975';

// Step 2: The SPARQL-resolved hierarchy is stored
// as OntologyConcept nodes with IS_A relationships
MERGE (c1:OntologyConcept {uri: 'icd10:G30.9'})
  SET c1.label='AD unspecified', c1.source_ontology='ICD-10';
MERGE (c2:OntologyConcept {uri: 'icd10:G30'})
  SET c2.label='Alzheimer disease', c2.source_ontology='ICD-10';
MERGE (c3:OntologyConcept {uri: 'icd10:G30-G32'})
  SET c3.label='Other degenerative diseases of NS',
      c3.source_ontology='ICD-10';
MERGE (c1)-[:IS_A]->(c2);
MERGE (c2)-[:IS_A]->(c3);

// Step 3: Bridge Diagnosis node to ICD-10 concept
MATCH (d:Diagnosis {icd10_code: 'G30.9'})
MATCH (c:OntologyConcept {uri: 'icd10:G30.9'})
MERGE (d)-[:CLASSIFIED_AS {source: 'ICD-10',
  resolved_via: 'SPARQL/BioPortal'}]->(c);
```

Thesis Value: This hybrid approach demonstrates mastery of both Neo4j and RDF/SPARQL. The SPARQL query happens *at enrichment time* (once), and the result is cached in Neo4j as OntologyConcept nodes. Runtime queries stay fast in Cypher. The SPARQL layer is consulted only when resolving new disease codes or traversing deep ICD-10 hierarchies.

9. Step-by-Step Implementation Plan — Revised

Phase 1: Schema Migration (LPG -> KG)

Step 1: Apply composite unique constraints and indexes (Section 6). This must happen BEFORE any data ingestion to prevent duplicates from the start.

Step 2: Add ontology properties to existing nodes (in-place upgrade). Run Cypher SET to add loinc_code, uberon_code, rxnorm_code to non-disease nodes. For Diagnosis nodes: add icd10_code, snomed_code, mondo_code.

Step 3: Resolve ICD-10 codes via SPARQL (Section 8). Query BioPortal for disease hierarchy. Store results as OntologyConcept nodes with IS_A. Create CLASSIFIED_AS edges from Diagnosis to ICD-10 concepts.

Step 4: Create OntologyConcept nodes for non-disease domains. Import ~200 concepts from SNOMED-CT, LOINC, UBERON, HPO. Build IS_A hierarchies in Neo4j.

Step 5: Create MAPS_TO relationships. Connect biomarker, assessment, brain region, medication nodes to OntologyConcept nodes.

Phase 2: Causal Discovery

Step 6: Extract flat feature matrix from KG. Cypher query pulls baseline data (MMSE, CDR, Abeta42, tau, ptau, APOE, ATN).

Step 7: Run causal discovery algorithms. PC, FCI (priority — handles latent confounders), GES, DAG-GNN. Compare outputs per Shen et al. (2020).

Step 8: Embed causal edges back into KG. CAUSES edges with algorithm, p_value, effect_size metadata.

Phase 3: Validation & Integration

Step 9: Validate against AlzKB and literature. Cross-reference causal edges. Mark as literature-validated.

Step 10: DoWhy causal inference + thesis defense preparation. Estimate effects, run refutation tests. Document Construct -> Discover -> Represent.

Key change from v1: Step 1 now prioritizes constraints BEFORE data insertion (was Step 4 in v1). Step 3 is new — dedicated SPARQL resolution for disease codes. Order matters: constraints first, then enrichment, then SPARQL.

10. Incremental Data Ingestion Pattern

Figure 5: MERGE-based upsert pipeline with hash-based change detection.

```
def ingest_observation(session, visit_id, table_name, row):
    """MERGE on composite key prevents duplicates."""
    row_hash = compute_row_hash(row, KEY_COLS[table_name])

    if table_name == 'MMSE':
        session.run("""
            MATCH (v:Visit {visit_id: $vid})
            MERGE (c:CognitiveAssessment {
                visit_id: $vid,
                test_name: 'MMSE'
            })
            ON CREATE SET
                c.total_score = $score,
                c.loinc_code = '72106-8',
                c.data_hash = $hash,
                c.created_at = datetime(),
                c.batch_id = $batch
            ON MATCH SET
                c.updated_at = CASE WHEN c.data_hash <> $hash
                    THEN datetime() ELSE c.updated_at END,
                c.total_score = CASE WHEN c.data_hash <> $hash
                    THEN $score ELSE c.total_score END,
                c.data_hash = $hash
            MERGE (v)-[:YIELDED_ASSESSMENT]->(c)
        """, vid=visit_id, score=row['MMSCORE'],
             hash=row_hash, batch=BATCH_ID)
```

11. AlzKB Integration Strategy

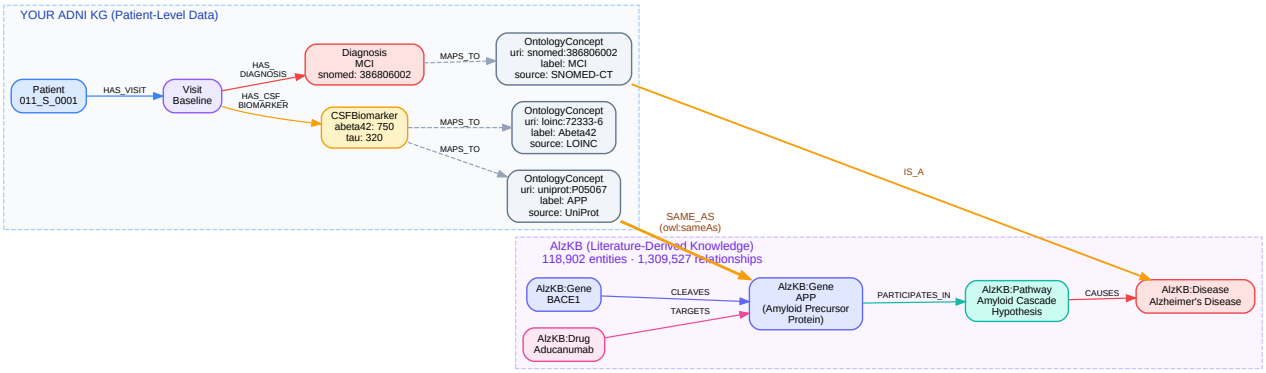

Figure 6: Bridge pattern — OntologyConcept nodes in your KG link to AlzKB entities via SAME_AS edges. Import only ~200-300 overlapping concepts.

AlzKB (Romano et al., 2024) contains 118,902 entities and 1,309,527 relationships. We import only concepts overlapping with ADNI data: AD-relevant genes (APOE, APP, PSEN1, MAPT), biomarker analytes, brain regions, and drugs patients take. SAME_AS links enable cross-graph traversal: Patient -> CSF -> MAPS_TO -> OntologyConcept -> SAME_AS -> AlzKB:Gene -> TARGETS -> AlzKB:Drug.

12. Mapping from headers.json to KG Nodes

KG Node	ADNI Tables	Key Columns
Patient	PTDEMOG, APOERES, Study_Entry, ARM	PTID, RID, PTGENDER, PTDOB, PTEDUCAT, APGEN1, APGEN2
Visit	All 108 tables (join key)	PTID, VISCODE/VISCODE2, VISDATE, PHASE, SITEID
Diagnosis	DXSUM, BLCHANGE	DXCURREN, DXCHANGE, DXCONV, DIAGNOSIS
CognitiveAssessment	MMSE, CDR, ADAS, MOCA, FAQ, GDSCALE, NPI/NPIQ, NEUROBAT, ECOG*, STAIAD (14 tables)	TOTSCORE, TOTAL13, CDGLOBAL, MMSCORE, etc.
CSFBiomarker	UPENNBIOMK_ROCHE_ELECSYS, BIOMARK, FOXLABBSI	ABETA42, ABETA40, TAU, PTAU, BATCH
BloodBiomarker	LABDATA, URMC_LABDATA, FNIHBC, JANSSEN	TestName, ResultValue, Units, Flags
MRIScan	Key_MRI, Structural_MRI_Images, MRI3META (7 tables)	image_id, modality, field_strength
PETScan	Key_PET, PET_Images, AV45META, TAUMETA (8 tables)	image_id, RADTRACER, CENTILOIDS
VolumetricMeasure	UCSFFSX7, UCBERKELEY_*, UCD_WMH, DTIROI_* (7 tables)	ST* (region volumes/ thickness/area)
GeneticProfile	APOERES, GENETIC	APGEN1, APGEN2, SNPs
Medication	BACKMEDS, RECCMEDS, ANTIAMYTX	Compound, drug class
AdverseEvent	ADVERSE, ADSXLIST, RECADV	AECOMPOUND, AEHSEVR
FamilyMember	FAMHXPAR, FAMHXSIB, FHQ, RECFHQ	Relationship, dementia hx
NeuropathFinding	NEUROPATH (164 cols)	Braak, CERAD, Thal, Lewy

Coverage: All 108 tables mapped. Study/Admin tables inform Visit and Patient properties. Other Assessment tables (24 tables) map to CognitiveAssessment with test_name distinguishing instruments.